# Redis

- In-memory data structure store, used most commonly as cache.
- Supports many DS like strings, hashes, lists, sets, sorted sets.
- Known for speed.

→ Uses

① caching objects to speed up web applications

Webserver → Redis → DB

Here, Redis stores frequently requested data in memory, so allows webservers to return freq. accessed data quick → reduces DB load and improves application response time.

At scale, cache distributed among cluster of Redis servers. Sharding is a common technique to distribute the caching load evenly across the cluster.

Things to consider here – a) setting a correct time-to-live (TTL)
b) thundering herd on cold start (empty cache + many requests hit DB simultaneously).

② used as session store to share session data among stateless servers

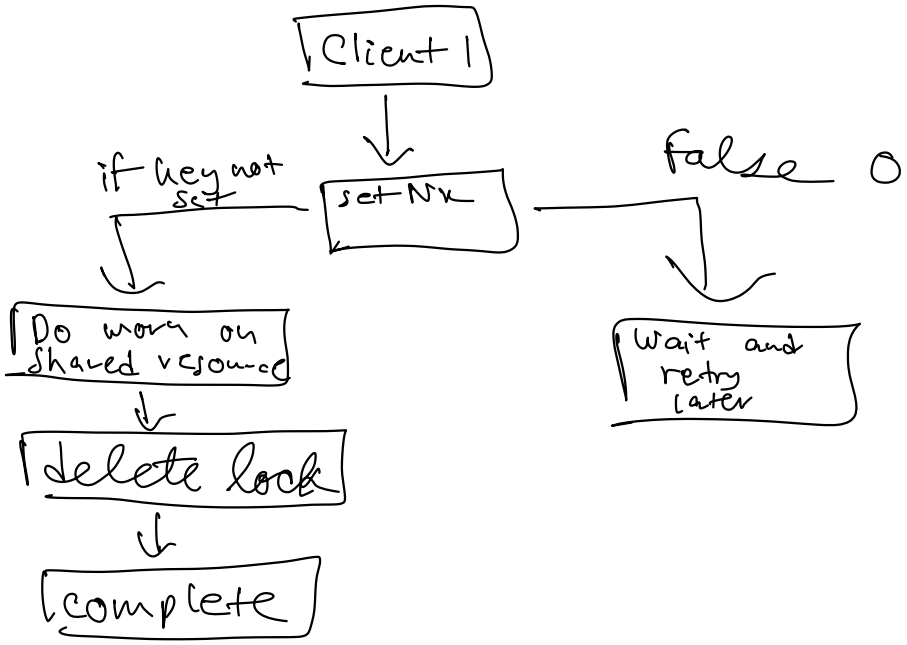
User login → session data stored in redis + unique session id that is returned to client as cookie

Redis is in-memory DB, so session data stored in redis lost if server restarts.

Though Redis provides persistence options like snapshots and AOF, these take too long to load to be practical. So, replication used instead usually.
↓
data replicated to backup instance, takes over in event of traffic.

## ③ Distributed lock

- Used when multiple nodes in an application need to coordinate access to some shared resource.
- allows caller to set a key only if it does not exist already.




## ④ Rate limiter

- Used as rate limiter by using increment command on counters and setting expiration times.

For each request, number of requests for key increased with INCR command in Redis. If curr_count in rate_limit processes Keys set to expire after specific time window to reset for next time window.

Leaky bucket algo can also be implemented with Redis.

## ⑤ Gaming Leaderboard

- uses sorted set

# RAG

**Retrieval Augmented Generation**

↓

LLMs generating text

Problems with LLMs
1. no source
2. out of date

adds content store
e.g. internet, document collection.

data ⇄ LLM ⇄ user

- data can be augmented (updated) to reflect newer data,
- less likely to hallucinate.

---

Upload → S3 (for storage) → Chunking Service → Embedding Service (converted into vector) → Vector DB

User Q → Embed Query → Similarity Search

↓

Top K Chunks + Question → LLM → Ans.

# Docker

- way to replicate the environment the software needs on any machine (regardless of OS, etc.).

→ VM :- an option but bully and slow as each runs its own OS.

→ Docker :- only virtualize OS rather than OS + hardware so all apps or containers run by single kernel

**Dockerfile**

↓

code that tells Docker how to build an image

**Image**

- snapshot of software
- immutable and can be used to spin up multiple containers

**Container**

- actual software running in real world

How it works :-

- create docker file
- use from to pull base image from cloud
- RUN, install dep, set env. variables
- create image file by running docker build command.

## AWS

- S3:- Giant folder in the cloud to which you upload files, each gets a URL you can use to access it later.
- EC2:- Computer in cloud you rent. Pick CPU and RAM you want, install code, it runs 24/7.
- Lambda:- function in cloud that only runs when something triggers it.
- SQS:- To do list for servers. Producer puts task on the list, consumer picks it up and does the work.
- RDS:- Cloud database that AWS manages for you

```
User uploads doc → S3 (stores file)
                 → SQS (queues a "process this doc" job)
                 → Lambda or EC2 worker picks up the job
                 → Chunks + embeds the doc
                 → Stores vectors in RDS (with pgvector
extension)
                 → User asks question → EC2 (FastAPI app) →
queries vectors → LLM → answer
```

## FastAPI

- Python framework to build APIs
- why this over others:-
  - type hints built in
  - auto generates API docs
  - async support (handles many requests simultaneously without blocking)
  - fast to build and run

## SQLAlchemy (ORM)

- Lets you define DB tables as python classes instead of raw SQL

```
User sends request → FastAPI (handles the endpoint)
                   → SQLAlchemy (reads/writes to database)
                   → RDS Postgres (stores the data)
```

## Cache Strategies

**Cache-Aside:** App checks the cache first. If data is there (cache hit), return it. If not (cache miss), get it from the database, put it in the cache, then return it. The app is responsible for managing the cache.

**Write-Through:** Every time your app writes data, it writes to BOTH the cache AND the database at the same time. This means the cache is always up-to-date. Reads are fast because the cache always has the latest data.

Downside: writes are slower because you're writing to two places every time. And you might be caching data that nobody ever reads — wasting memory.

**Write-Back (also called Write-Behind):** Your app writes to the cache ONLY. The cache then updates the database later, in the background, maybe every few seconds or in batches.

Writes are super fast because you're only writing to memory. But there's a risk: if the cache crashes before it writes to the database, you lose data.

**When to use each — the interview-level summary:**

- **Cache-aside:** most common, good default choice. Use when reads are much more frequent than writes. "I'd use cache-aside with Redis and a 5-minute TTL."
- **Write-through:** use when you need the cache to always be consistent with the database. Slower writes but no stale data.
- **Write-back:** use when write speed is critical and you can tolerate a small risk of data loss. Good for things like analytics counters or logging where losing a few data points isn't catastrophic.

**For sant.ai specifically:** if they ask about caching in the workflow engine or RAG system, say "I'd use cache-aside with Redis for caching frequent queries, with a TTL to handle cache invalidation." That's the safe default answer.